Light

GCSE Computer Science - Searching and Sorting Algorithms (Set 1)

Over **1,100,000+ marks** of exam-aligned content across **67 qualifications**. 4,200+ auto-marked assessments & long-answer practice. No login required.

GCSE • OCR J277 • Grades 3—9 | Topic test | 30 min | 55 marks | Answered: 0 / 30

00:00 Start Pause Reset

← Back to Algorithms Searching Sorting

Multiple-choice topic test for **GCSE Computer Science (OCR J277)**. You may skip questions. When you submit, you'll see your score, a grade estimate, and worked explanations. **Tip:** use the stopwatch above to time yourself.

Q1 (1 mark)

What is a linear search?

- ○ **A.** An algorithm that checks each item in a list one by one from the beginning until the target is found or the end is reached
- ○ **B.** An algorithm that only works on sorted data
- ○ **C.** An algorithm that always finds items instantly
- ○ **D.** An algorithm that sorts data before searching

Q2 (1 mark)

What is a pre-requisite for using a binary search algorithm?

- ○ **A.** The data must contain only numbers
- ○ **B.** The data must be stored in a two-dimensional array
- ○ **C.** The data must be sorted in order
- ○ **D.** The data must contain fewer than 100 items

Q3 (1 mark)

How does bubble sort work?

- ○ **A.** It randomly shuffles items until they happen to be in order
- ○ **B.** It compares adjacent pairs of items and swaps them if they are in the wrong order, repeating until the list is sorted
- ○ **C.** It finds the smallest item and moves it to the start, then repeats for remaining items
- ○ **D.** It splits the list into halves and sorts each half separately

Q4 (1 mark)

Which sorting algorithm splits data into individual items before recombining them in order?

- ○ **A.** Bubble sort
- ○ **B.** Insertion sort
- ○ **C.** Linear sort
- ○ **D.** Merge sort

Q5 (1 mark)

How does insertion sort work?

- ○ **A.** It inserts new items at random positions
- ○ **B.** It deletes items and reinserts them elsewhere
- ○ **C.** It builds a sorted portion of the list one item at a time, inserting each new item into its correct position within the sorted portion
- ○ **D.** It inserts dividers between groups of items

Q6 (1 mark)

What is the main advantage of linear search over binary search?

- ○ **A.** Linear search works on unsorted data, while binary search requires sorted data
- ○ **B.** Linear search is always faster
- ○ **C.** Linear search uses less memory
- ○ **D.** Linear search can only find one item at a time

Q7 (1 mark)

In binary search, what happens after comparing the target to the middle element?

- ○ **A.** The search always moves to the left half
- ○ **B.** The search ends immediately

- ○ **C.** If the target is smaller, search the left half; if larger, search the right half; if equal, the item is found
- ○ **D.** The middle element is deleted from the list

Q8 (1 mark)

After one complete pass of bubble sort on an unsorted list, what is guaranteed?

- ○ **A.** The entire list will be sorted
- ○ **B.** The smallest item will be at the start
- ○ **C.** The largest item will have moved to its correct position at the end
- ○ **D.** Exactly half the items will be in the correct position

Q9 (1 mark)

For a sorted list of 1,000 items, approximately how many comparisons would binary search need in the least efficient scenario?

- ○ **A.** 1,000 comparisons
- ○ **B.** 500 comparisons
- ○ **C.** About 10 comparisons
- ○ **D.** 1 comparison

Q10 (1 mark)

What does it mean when we say a sorting algorithm sorts data 'in ascending order'?

- ○ **A.** Items are arranged from smallest to largest
- ○ **B.** Items are arranged from largest to smallest
- ○ **C.** Items are arranged alphabetically only
- ○ **D.** Items are arranged randomly

Q11 (1 mark)

What is the 'merge' step in merge sort?

- ○ **A.** Randomly combining two lists together
- ○ **B.** Deleting duplicate items from the list
- ○ **C.** Splitting a list into two equal halves
- ○ **D.** Combining two sorted sub-lists into one larger sorted list by comparing elements from each

Q12 (1 mark)

In the least efficient scenario, how many comparisons does linear search need for a list of n items?

- ○ **A.** 1 comparison
- ○ **B.** n/2 comparisons
- ○ **C.** n comparisons

- **D.** n squared comparisons

Q13 (2 marks)

A binary search is used to find the value 7 in this sorted list:
[1, 3, 5, 7, 9, 11, 13]
How many comparisons are needed?

- **A.** 1 comparison - 7 is found immediately
- **B.** 2 comparisons - check middle (7), found
- **C.** 3 comparisons - check 7, then 3, then 9
- **D.** 7 comparisons - every item must be checked

Q14 (2 marks)

The following list is being sorted using bubble sort in ascending order:
[5, 2, 8, 1]
What will the list look like after the first complete pass?

- **A.** [2, 5, 1, 8]
- **B.** [1, 2, 5, 8]
- **C.** [2, 1, 5, 8]
- **D.** [5, 2, 1, 8]

Q15 (2 marks)

When sorting the list [6, 2, 8, 4] using merge sort, what are the sub-lists after the first level of division?

- ○ **A.** [6] and [2, 8, 4]
- ○ **B.** [6, 2, 8] and [4]
- ○ **C.** [2, 4, 6, 8] (already merged)
- ○ **D.** [6, 2] and [8, 4]

Q16 (2 marks)

The list [3, 1, 4, 2] is being sorted using insertion sort. After the second item has been processed, what does the list look like?

- ○ **A.** [3, 1, 4, 2] - unchanged
- ○ **B.** [3, 4, 1, 2] - 4 moved forward
- ○ **C.** [1, 2, 3, 4] - fully sorted
- ○ **D.** [1, 3, 4, 2] - 1 inserted before 3

Q17 (2 marks)

A phone app needs to search a contact list that users can add to at any time (contacts are not automatically sorted). Which search algorithm should be used?

- ○ **A.** Binary search, because it is faster
- ○ **B.** Bubble search, because it works with contact names
- ○ **C.** Merge sort, because it can search and sort simultaneously
- ○ **D.** Linear search, because the data is not guaranteed to be sorted

Q18 (2 marks)

How can bubble sort detect that a list has become fully sorted before completing all possible passes?

- ○ **A.** It cannot - bubble sort always performs the same number of passes regardless of input
- ○ **B.** By checking if the first and last elements are in order
- ○ **C.** By tracking whether any swaps were made during a pass - if no swaps occur, the list is sorted
- ○ **D.** By comparing the list to a backup copy made at the start

Q19 (2 marks)

Why does merge sort require additional memory compared to bubble sort?

- ○ **A.** Merge sort stores a backup copy of the original list for safety
- ○ **B.** Bubble sort compresses data to use less memory
- ○ **C.** Merge sort uses memory to count the number of comparisons made
- ○ **D.** Merge sort needs extra space to hold the sub-lists during the divide and merge process

Q20 (2 marks)

Linear search is used to find if a student name exists in a class register array. The function should return the index position if found, or -1 if not found.
If the class has 25 students and the searched name is at position 10, how many comparisons are made?

- ○ **A.** 25 comparisons - the entire list is always checked
- ○ **B.** 10 comparisons - the search stops when the name is found at position 10
- ○ **C.** 11 comparisons - positions 0 to 10 inclusive are checked
- ○ **D.** 12 comparisons - positions 0 to 10 plus one extra to confirm

Q21 (2 marks)

For a list that is already nearly sorted (only a few items out of place), which sorting algorithm would likely perform best?

- ○ **A.** Merge sort, because it always performs consistently
- ○ **B.** Insertion sort, because it efficiently handles nearly-sorted data with minimal operations
- ○ **C.** Bubble sort with no optimisation, because it's the simplest
- ○ **D.** Binary sort, because it can find where items belong quickly

Q22 (2 marks)

Binary search is used to find 3 in the sorted list [1, 2, 3, 4, 5, 6, 7, 8, 9]. Which elements are compared to the target before 3 is found?

- ○ **A.** 5 only, then 3 is found
- ○ **B.** 5, then 2, then 3 is found
- ○ **C.** 1, then 2, then 3 is found (like linear search)
- ○ **D.** 5, then 3 is found immediately

Q23 (2 marks)

When bubble sort is used on the list [4, 3, 2, 1] (sorting to ascending order), how many swaps occur in the first pass?

- ○ **A.** 0 swaps - the list is already in order
- ○ **B.** 1 swap - only adjacent pairs out of order are swapped
- ○ **C.** 3 swaps - every adjacent pair needs swapping
- ○ **D.** 6 swaps - some pairs are swapped multiple times

Q24 (3 marks)

A library database contains 1,000,000 book records sorted alphabetically by title. A librarian needs to check if a specific title exists. Compare the efficiency of linear search versus binary search for this task.

- ○ **A.** Both algorithms would check approximately the same number of records
- ○ **B.** Linear search: up to 1,000,000 comparisons in least efficient scenario. Binary search: about 20 comparisons in least efficient scenario (halving 1,000,000 repeatedly reaches 1 after about 20 steps). Binary search is dramatically more efficient for large sorted datasets because each comparison eliminates half the remaining possibilities.
- ○ **C.** Linear search would be faster because it's simpler
- ○ **D.** Binary search cannot work on text data like book titles

Q25 (3 marks)

Trace through merge sort on the list [8, 3, 5, 1].
Show the complete divide and merge process.

- ○ **A.** The list cannot be sorted using merge sort
- ○ **B.** Divide: [8,3,5,1] then [8,3] and [5,1] then [8][3][5][1]. Merge: [3,8] and [1,5] then [1,3,5,8]. The algorithm first splits until single elements, then merges pairs in order, building larger sorted sub-lists until complete.
- ○ **C.** The list is sorted in one step by comparing all elements simultaneously
- ○ **D.** Divide: [8,3,5,1] then [8][3,5,1]. This uneven split continues until sorted.

Q26 (3 marks)

Trace insertion sort on [5, 2, 4, 1, 3], showing the state of the list after each insertion.

- ○ **A.** After each: [2,5,4,1,3] then [2,4,5,1,3] then [1,2,4,5,3] then [1,2,3,4,5]. Each new element is inserted into its correct position within the growing sorted portion.
- ○ **B.** [5,2,4,1,3] then [1,2,3,4,5] in one step
- ○ **C.** [2,5,4,1,3] then [2,4,5,1,3] then [2,4,1,5,3] then [2,1,4,5,3] then [1,2,4,5,3]
- ○ **D.** The list cannot be sorted using insertion sort

Q27 (3 marks)

A company needs to sort customer records for their database. The data changes frequently (new customers added daily) and quick individual lookups are needed.
Analyse which combination of sorting and searching algorithms would be most appropriate.

- ○ **A.** Never sort the data; always use linear search for every lookup
- ○ **B.** Sort once using any algorithm; never update the sorted order
- ○ **C.** Use insertion sort to maintain order when adding new records (efficient for adding single items to sorted data), combined with binary search for quick lookups. This balances efficient insertion of new data with fast searching of the sorted database.
- ○ **D.** Use merge sort after every single lookup to ensure data is sorted

Q28 (3 marks)

Consider an optimised bubble sort that tracks swaps and stops early when the list is sorted. Compare its performance on these two inputs of length 100:
Input A: [1, 2, 3, ..., 99, 100] (already sorted)
Input B: [100, 99, 98, ..., 2, 1] (reverse sorted)

- ○ **A.** Both inputs require the same number of passes because bubble sort always does n-1 passes
- ○ **B.** Input A: 1 pass with 0 swaps detects it's sorted, stops immediately. Input B: requires 99 passes with maximum swaps each pass (the least efficient scenario). Optimised bubble sort benefits greatly from favourable input but still suffers on least efficient scenario input.
- ○ **C.** Input A requires more passes because larger numbers need to bubble further
- ○ **D.** Input B would be faster because swapping is faster than not swapping

Q29 (3 marks)

A spell-checker needs to determine if words typed by a user exist in a dictionary of 50,000 words stored in alphabetical order.

Analyse the most efficient approach for this application.

- ○ **A.** Load words into memory unsorted and use linear search for maximum flexibility
- ○ **B.** Use binary search on the sorted dictionary data. With 50,000 words, binary search needs at most 16 comparisons (halving 50,000 repeatedly reaches 1 after about 16 steps) versus up to 50,000 for linear search. For frequent lookups during typing, binary search is approximately 3,000 times faster on average.
- ○ **C.** Sort the dictionary each time a word is looked up to ensure freshness
- ○ **D.** Linear search would be faster because alphabetical order doesn't help with spelling

Q30 (3 marks)

A streaming music service needs to merge two already-sorted playlists (one with 1,000 songs, another with 500 songs) into one combined sorted playlist.
Analyse the most efficient approach.

- ○ **A.** Put all 1,500 songs in one list and use bubble sort to sort them from scratch
- ○ **B.** Use only the merge step from merge sort. Since both playlists are already sorted, merge them directly by comparing front elements and taking the smaller. This requires at most 1,500 comparisons, far fewer than re-sorting from scratch which could require millions of comparisons.
- ○ **C.** Use binary search to find where each song from the smaller playlist should be inserted
- ○ **D.** Randomly shuffle both playlists together and hope they end up sorted

Total: **55** marks • Time: **30 min**

↑ Back to top

Submit answers

Williams Physics Education — over **1,100,000+ marks** of exam-aligned content and **4,200+ auto-marked assessments** across 67 qualifications at GCSE, A-Level, AP & Professional Qualifications. Free to use, no login required.

About & Contact • Privacy • Terms • Accessibility • Copyright • DMCA • YouTube

WPE is free for everyone. If our resources help you or your students, you can support the project here.

© 2026 Williams Physics Education. All rights reserved.